Open Access

Building the Data Model: From Raw Data to Analysis-Ready Tables

LS100 — Module 00A · Research Plans & Proposals

Last updated: 2026-09-30

Authored by **Souvik Mandal, Ph.D.**

Project Leader & Instructor, Computational Behavioral Sciences, LS100, FAS, Harvard University | LinkedIn ID: souvik-mandal-phd

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-00a-research-plans-and-proposals/

Published Oct 15, 2026

A **data model** is the blueprint of how your data is organized at every step between the sensor and the statistical test. *Converting the Problem to a Research Framework* (Section 5) explains why you need one. This guide is the procedure for building it.

1. The three stages of data

Most of the research projects in this course move data through the same three stages:

Stage	What it is	One row is
Primary	What the sensor recorded (video, audio, logs). Never edited.	not a table — one file per recording
Secondary	What a model or algorithm extracts from one primary file.	one unit of extraction (a frame, a time window, an event)
Tertiary	Analysis-ready variables derived from the secondary data.	one unit of analysis

Choose the row meaning for the tertiary table from your research question. In the secondary table, one row might represent one detected frame. If your analysis compares participants' average knee position, each tertiary row represents one participant: group the frame-level data by `participant_id` and calculate the participant's mean knee x- and y-coordinates. If your analysis compares trials instead, each tertiary row represents one trial.

It is a good practice to describe this flow in three passes, each more concrete than the last: **conceptual** (a diagram), **logical** (table schemas), and **physical** (folders, files, and Python).

2. Assignment 02: What you hand in

One Jupyter notebook that contains the diagram, the schemas, and the folder layout in Markdown cells, followed by code that takes **one** primary file and produces its **secondary dataframe**. Producing the tertiary dataframe as well earns a bonus.

3. Pass 1 — Conceptual: draw the flow

No technical detail yet. Name the **entities** your study tracks (e.g., participant, session, trial, recording, etc.) and draw how data moves between stages. Every arrow must be labelled with the transformation that performs it.

```
[Entity: participant → session → trial]
              | record
              ▼
      PRIMARY: one raw file per trial
              | extract  (model / algorithm)
              ▼
      SECONDARY: one table per raw file
              | derive + aggregate  (your code)
              ▼
        TERTIARY: one table/dataframe per unit of analysis in analysis-
ready format
              |
              ▼
        All the data files of the whole study
              |
              ▼
      statistical analysis
```

If you cannot name the transformation on an arrow, you have found a gap in your plan.

4. Pass 2 — Logical: write the table schemas

A **schema** is a plan for a table: it lists each column and describes its data type, unit, role, and source. Write one schema for the secondary table and one for the tertiary table, independent of any software or programming language. Each column gets one line.

For this pose-estimation example, assume the pipeline reports pixel coordinates in the original video frame. In the secondary table, one row is one extracted frame:

Column name	Type	Unit	Role	Comes from
`participant_id`	text	—	ID	file name
`session_id`	text	—	ID	file name
`trial_id`	text	—	ID	file name
`frame_index`	integer	—	ID	extraction
`knee_x_coordinate`	float	pixels	measure	pose-estimation pipeline
`knee_y_coordinate`	float	pixels	measure	pose-estimation pipeline

In the tertiary table, one row is one participant:

Column name	Type	Unit	Role	Comes from

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-00a-research-plans-and-proposals/

Column name	Type	Unit	Role	Comes from
`participant_id`	text	—	ID	grouping key in secondary table
`mean_knee_x_coordinate`	float	pixels	measure	mean of secondary `knee_x_coordinate` values for that participant
`mean_knee_y_coordinate`	float	pixels	measure	mean of secondary `knee_y_coordinate` values for that participant

This participant-level summary uses frames from all of that participant's recordings. Producing it requires combining secondary tables; the required assignment only asks you to process one primary file.

Three rules:

1. **IDs match the row meaning.** A secondary row might include `participant_id`, `session_id`, and `trial_id`, plus its `frame_index`. A participant-level tertiary row includes `participant_id`; a trial-level tertiary row includes the IDs needed to identify that trial (for example, `participant_id`, `session_id`, and `trial_id`). If you need to trace an aggregate back to its contributing files, record that provenance separately.
2. **One row, one meaning.** State it in a sentence under each schema: "one row is one _____."
3. **Every column has a source.** A tertiary column names the secondary columns and the formula that produce it. A column with no source cannot be computed.

Use a separate metadata table for each entity level you record: `participants.csv` has one row per participant, and `sessions.csv` has one row per session. Include the ID for that entity in each table and join it to the data using that ID.

5. Pass 3 — Physical: folders, file names, and Python

Folders. One folder per stage; data only moves downward.

```
project/
├── data/
│   ├── 01_primary/      # raw files, read-only
│   ├── 02_secondary/    # one table per raw file
│   ├── 03_tertiary/     # one analysis-ready table
│   ├── participants.csv
│   └── sessions.csv
└── notebooks/
```

File names. Encode the IDs in the name, in a fixed order, separated by underscores: `<participant>_<session>_<trial>.<ext>`. No spaces, no "final", no dates typed by

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-00a-research-plans-and-proposals/

hand. A secondary file keeps the name of the primary file it came from, so the link between them never depends on memory.

Python. In your notebook, use your Pass 2 schema to create a secondary dataframe from one primary file, check that its columns match the schema, and save it under `data/02_secondary/`. For the extraction step, adapt the course notebooks: pose estimation from one video for video data, and audio data foundation for audio data.

Bonus — tertiary. Combine secondary dataframes as needed for your analysis unit, compute the derived variables in your tertiary schema, and save the result to `03_tertiary/`.

6. Notebook rules

- Every code cell is preceded by a Markdown cell that states **why** the step is needed and **what** the code does. Write it for a classmate who has not seen your project.
- The notebook runs top to bottom on a fresh kernel, from one raw file, without manual edits between cells.
- Paths are relative to the project folder; nothing points to your personal machine.
- The last cell displays `secondary.head()` and `secondary.shape`, so the output can be checked against your schema at a glance.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-00a-research-plans-and-proposals/